ONSITE TECHNICAL TASK · 3 HOURS

Proctored Application & CV Integrity System

Build a small, locally-running system that intakes a job applicant, runs a step-wise questionnaire, and uses computer vision to assess the integrity of their responses in real time.

ROLE	TIME BOX	RUNS	TOOLS
AI Engineer	**3 hours**	**Fully local**	**Any / AI-assisted**

1 · Context

We hire remotely and run early-stage screening questionnaires that candidates complete on their own. We want a prototype of a proctoring layer that sits on top of an applicant-intake flow (a lightweight CRM) and flags integrity concerns — a candidate looking off-screen for long stretches, pasting answers rather than typing them, or leaving the frame — so a human reviewer can triage sessions faster.

This is a scoped prototype, not a production system. We care far more about how you reason about an ambiguous problem, structure a CV pipeline, and make sensible trade-offs under a time box than about polish or exhaustive coverage. Use whatever tools accelerate you — Claude Code, Cursor, Google AI Studio, Copilot. You will walk us through and defend the result live.

2 · What you'll build

A single locally-runnable web app with three connected parts:

A

Applicant intake (CRM)

Capture applicant details and start a session.

B

Step-wise questionnaire

One question at a time, with typed answers.

C

CV integrity monitor

Camera-based signals + a review summary.

Part A — Applicant intake

- A form collecting **Name**, **Email**, and **Role** (a dropdown — e.g. Software Engineer, Designer, DevOps, Data Scientist, Product Manager).
- Basic validation (required fields, valid email). On submit, create a session record and proceed to the questionnaire.
- Persist the applicant + session somewhere inspectable (SQLite, JSON file, in-memory store — your call). We should be able to see the record afterward.

Part B — Step-wise questionnaire

- Present questions **one at a time** (a wizard/stepper), with progress indication and next/back navigation. 3–5 questions is plenty; content can be generic.
- Answers are free-text. Capture enough interaction telemetry per answer to support Part C (see below).
- Camera access must be requested and the webcam must be live for the duration of the questionnaire; block progression if the camera is denied or unavailable, with a clear message.

Part C — CV integrity monitor

Using the live camera feed and the input telemetry, detect and log the following signals. Aim to implement the first three well rather than all of them shallowly.

- **Gaze / attention** — flag when the applicant is looking away from the screen (head pose or gaze estimation) for a sustained period.
- **Paste vs. genuine typing** — distinguish pasted text and implausibly fast bulk input from real keystroke-by-keystroke typing.
- **Presence** — flag when no face is detected (candidate left frame) or when more than one face appears.
- **Optional signals** — tab/window blur, a phone or second screen visible, or prolonged silence. Treat as stretch.

> **Output that matters**
>
> Each session ends with a reviewable **integrity summary**: per-signal counts, a timeline or list of flagged events (with timestamps and, where relevant, the question they occurred on), and an overall confidence/risk indication. A live indicator during the questionnaire is a plus. Do not auto-reject anyone — this assists a human, it does not replace one.

3 · Constraints & ground rules

- **Runs locally.** We must be able to clone/open and run it on one machine with a documented set of steps. No mandatory paid cloud services.
- **Any stack.** Language, framework, and CV approach are your choice — browser-side (MediaPipe, TensorFlow.js) or a Python backend (OpenCV, MediaPipe, dlib) both fine. Justify the choice.
- **AI tooling encouraged.** Claude Code, Cursor, Google AI Studio, Copilot — use them. But you own every line: you must be able to explain, modify, and debug it live.
- **Privacy.** Keep video processing local; don't stream raw video off the machine. Note what you store and for how long.
- **Scope honestly.** A smaller system that works and that you understand deeply beats a large one you can't defend.

4 · Deliverables

- A **working local demo** we can run during the session.
- The **source** (git repo or folder), with a short **README**: how to run it, the stack, and known limitations.
- A brief **design note** (README section or a page): key decisions, what each CV signal actually measures, false-positive risks, and what you'd do with more time.

Submission

Push your work to a **public repository** and submit the link via this form: **forms.gle/UPskS9iAV6iQ3Hnd6**. Submit before the walkthrough begins.

5 · Suggested time allocation

PHASE	FOCUS	BUDGET
Setup & scaffolding	Project skeleton, intake form, routing	~30 min
Questionnaire + camera	Stepper, telemetry capture, webcam live	~45 min
CV signals	Gaze, paste-detection, presence	~75 min
Summary + README	Integrity report, docs, cleanup	~30 min

6 · Evaluation rubric

Scored out of 100. We weight reasoning and engineering judgment above feature count. A partial system with sharp thinking scores well.

CRITERION	WHAT WE LOOK FOR	WT.
CV pipeline quality	Sound approach to gaze/presence/paste detection; sensible model/library use; handles noise.	**30**
Functionality & flow	Intake → questionnaire → camera → summary works end-to-end without hand-holding.	**20**
Code quality & structure	Readable, modular, sensible state/data handling; not a single 1000-line file.	**15**
Trade-off reasoning	Clear rationale for stack, signals, and thresholds; awareness of false positives & limits.	**15**
Code walkthrough & Q&A	Can explain, navigate, and modify the code live; understands the AI-generated parts.	**10**
Product & ethics sense	Treats flags as assistive, considers privacy, bias, and fairness of proctoring.	**10**

Signals of a strong submission

- Debounced flags, not per-frame noise
- Named thresholds with a rationale
- Graceful camera-denied / no-face handling
- Honest README on what doesn't work yet

Red flags

- Can't explain their own AI-generated code
- Auto-rejects candidates from raw flags
- Streams raw video to a third party
- Demo doesn't run without live debugging

7 · Live walkthrough & Q&A

After the build, expect ~20–30 minutes of discussion. Be ready to:

- Run the demo and walk us through the flow and the codebase.
- Explain any AI-assisted section as if you'd written it by hand — why it works, where it breaks.
- Answer fundamentals: how your gaze/head-pose estimation works, why paste detection is reliable or not, event debouncing, and how you'd reduce false positives.

- Discuss scaling it up: many concurrent sessions, model choice, storage, and the ethics of automated proctoring.

Questions? Ask your interviewer before you start — clarifying scope is expected, not penalized.